

Université Hassan 1^{er}
Faculté des Sciences et Techniques
Département Mathématiques et Informatique
Filière : GI -2020/2021



La programmation en langage Swift

Préparé par : Y.BLOUKI

Introduction

Swift est un nouveau langage de programmation pour les applications iOS, OS X, watchOS et tvOS conçu par Apple Inc., et a été initialement mis à la disposition des développeurs Apple en 2014, principalement destiné à remplacer le Langage Objective-C qui était à la base du développement de logiciels OS X et iOS à l'époque. Swift propose un modèle de programmation sûr et sécurisé, ajouté à des fonctionnalités modernes ayant pour but de rendre la programmation plus simple, plus flexible et plus fun. Avec l'aide de Cocoa et Cocoa Touch, des frameworks matures et populaires, Swift offre une opportunité pour réinventer le développement logiciel. Il représente un moyen fantastique d'écrire des logiciels, que ce soit pour les téléphones, les ordinateurs de bureau, les serveurs ou tout autre élément qui exécute du code. C'est un langage de programmation sûr, rapide et interactif.

Contrairement à de nombreux langages orientés objet, qui sont basés sur des langages procéduraux plus anciens-par exemple, C ++ et Objective-C sont basés sur C — Swift a été conçu à partir de zéro en tant que nouveau langage moderne et orienté objet qui rend la programmation plus rapide et plus facile, et aide les développeurs à produire un code expressif moins sujet aux erreurs que de nombreux langages. Le compilateur est optimisé pour les performances et le langage est optimisé pour le développement, sans compromettre ni l'un ni l'autre.

Swift définit plusieurs classes d'erreurs de programmation courantes en adoptant des modèles de programmation modernes:

- Les variables sont toujours initialisées avant utilisation.
- Les indices de tableau sont vérifiés pour les erreurs hors limites.
- Les nombres entiers sont vérifiés pour le dépassement.
- Les options garantissent que les **nil** valeurs sont gérées explicitement.
- La mémoire est gérée automatiquement.
- La gestion des erreurs permet une récupération contrôlée après des échecs inattendus.

Bien qu'il n'est pas basé sur un langage plus ancien, Swift, selon les mots de son architecte, Chris Lattner, "s'est inspiré d'Objective-C, Rust, Haskell, Ruby, Python, C #, CLU, et d'autres."

Swift reste principalement utilisé par les développeurs ciblant les Plateformes Apple macOS et iOS, Swift est entièrement pris en charge sous Linux, et d'autres efforts communautaires sont en cours pour prendre en charge Android, Windows et d'autres plates-formes.

L'objectif de ce cours est d'apprendre les fondements de l'utilisation du langage de programmation Swift, la syntaxe de base et la structure du programme Swift. Vous apprendrez également l'utilisation des types de données et les énumérations intégrés, et comment déclarer et utiliser les variables et les constantes en Swift.

I. Structure du programme Swift

, Nous examinerons, dans cette première section la syntaxe basique du langage Swift, et vous écrirez votre premier programme Swift entièrement fonctionnel.

Comme de nombreux langages de programmation modernes, Swift tire sa syntaxe la plus basique du langage de programmation C. Si vous avez une expérience de programmation dans d'autres Langages inspirés du C, tels que C++, Java, C #, Objective-C ou PHP, de nombreux aspects de Swift vous semblera familier, et de nombreux concepts Swift vous seront probablement familiers. Nous pouvons dire à propos la syntaxe de base du Swift que:

- Les programmes sont constitués d'instructions, exécutées séquentiellement
- Plusieurs instructions sont autorisées par ligne d'éditeur lorsqu'elles sont séparées par un point-virgule (;)
- Les unités de travail dans Swift sont modulaires à l'aide de fonctions et organisées en types
- Les fonctions acceptent un ou plusieurs paramètres et renvoient des valeurs
- Les commentaires simples et multilignes suivent la même syntaxe qu'en C et Java
- Les noms et l'utilisation des types de données Swift sont similaires à ceux de Java, C # et C++
- Swift a le concept de variables, qui sont mutables, et de constantes, qui sont immuables
- Swift a à la fois une sémantique de struct et de classe, tout comme C et C #

Swift présente des améliorations et des différences par rapport aux langages Java, C # ou C++ :

- Les points-virgules ne sont pas obligatoires à la fin des instructions, sauf lorsqu'ils sont utilisés pour séparer plusieurs instructions tapées sur la même ligne dans un fichier source.
- Swift n'a pas de méthode `main ()` pour servir de point de départ du programme lorsque le système d'exploitation charge l'application. Les programmes Swift commencent à la première ligne de code du fichier source du programme - comme c'est le cas dans la plupart des langages interprétés.
- Les fonctions dans Swift placent le retour de fonction sur le côté droit de la fonction déclaration, plutôt que la gauche.
- La syntaxe de déclaration des paramètres de fonction est inspirée d'Objective-C, qui est assez différent et souvent déroutant au début pour les développeurs Java, C # et C ++.
- La différence entre une structure et une classe dans Swift est similaire à ce qui a été défini en C # (type valeur versus type référence).

La tradition suggère que le premier programme dans un nouveau langage devrait afficher les mots "Hello, world!" sur l'écran. En Swift, on peut le faire en une seule ligne :

```
print("Hello, world!")
```

En Swift, cette ligne de code est un programme complet. Vous n'avez pas besoin d'importer une bibliothèque séparée pour des fonctionnalités comme la gestion des entrées/sorties ou des chaînes de caractère. Le code écrit dans le champ global est utilisé comme le point d'entrée du programme, ce qui signifie que vous n'avez pas besoin d'une fonction `main()` . Par ailleurs, les points-virgules à la fin de chaque ligne ne sont pas obligatoires.

Vous commencez par l'utilisation du **playground** du **Xcode** pour créer un programme Swift simple pour afficher la chaîne "Hello, world!" à la console du **playground**, en suivant ces étapes:

1. lancez Xcode. Vous recevrez un « **Welcome to Xcode** » écran avec les commandes suivantes répertoriées à gauche:

1. **Get started with a playground**
2. **Create a new Xcode project**

3. Clone an existing project

Puisque nous allons écrire du code mais ne pas créer une application, choisissez l'option **playground** pour ouvrir une fenêtre de code interactive.

2. Choisissez **Blank** comme template du **playground**, puis appuyez sur le bouton Suivant.

3. Ensuite, Xcode vous demandera là où il va enregistrer le **playground**. Cela enregistrera votre code dans un fichier avec une extension **playground**.

5. Lorsque Xcode crée un nouveau **playground**, il ajoute du code par défaut. Appuyez sur ⌘A puis sur la touche Suppr du clavier pour supprimer l'exemple de code.

6. Dans l'éditeur, ajoutez les deux lignes de code suivantes:

```
let message = "Hello, World."  
print(message)
```

II. Variables et Constantes Swift

Tous les langages de programmation permettent aux programmeurs de stocker des valeurs dans la mémoire en utilisant un nom associé choisi par le programmeur. Les variables permettent aux programmes d'opérer sur des valeurs de données qui changent pendant l'exécution du programme.

Utilisez **let** pour créer une constante et **var** pour créer une variable. La valeur d'une constante n'a pas besoin d'être définie au moment de la compilation, mais sa valeur ne peut être définie qu'une seule et unique fois. Ainsi, vous pouvez utiliser des constantes pour nommer une valeur qui ne sera déclarée qu'une seule fois mais que vous réutiliserez à plusieurs endroits.

La déclaration des variables Swift utilise la syntaxe de base suivante:

```
var <nom de la variable>: <type> = <valeur>
```

Compte tenu de cette syntaxe, une déclaration légale pour une variable **maVariable** serait la suivante:

```
var maVariable : Double = 6,14
```

Cette déclaration signifie: créer une variable nommée **maVariable**, qui stocke un type de données Double, et lui attribuer une valeur initiale de 6,14.

La syntaxe de déclaration des constantes Swift est comme suit:

```
let <nom de la constante>: <type> = <valeur>
```

Exemple : `let pi = 3.14`

Une constante ou une variable doivent avoir le même type que celui de la valeur que vous voulez lui donner. Toutefois, il n'est pas toujours nécessaire d'écrire le type explicitement. Donner une valeur à une constante ou une variable quand vous la déclarez permet au compilateur de déduire son type. Dans l'exemple ci-dessus, le compilateur a déduit que `pi` est de type "Double" car sa valeur initiale est un double (3.14). Si la valeur initiale ne fournit pas assez d'informations (ou si il n'y a pas de valeur initiale), spécifiez le type de la variable/constante en l'écrivant après le symbole : .

Note : Hormis la restriction sur la mutation de la valeur d'une constante une fois déclarée (pour des raisons de sécurité), les variables et les constantes Swift sont utilisées de manière pratiquement identique.

Inférence de type

Dans l'exemple précédent, nous avons créé la variable `pi` sans spécifier son type de données. Nous avons profité d'une fonctionnalité du compilateur Swift appelée inférence de type. Lorsque vous affectez la valeur d'une variable ou d'une constante lors de sa création, le compilateur Swift analyse le côté droit de l'affectation, déduit le type de données et attribue ce type de données à la variable ou à la constante que vous avez créé. Par exemple, dans la déclaration suivante, le compilateur créera le nom de la variable en tant que type de données `String` :

```
var name = "Ahmedi"
```

Comme Swift est un langage sécurisé, une fois qu'un type de données est inféré par le compilateur, il reste fixe pour la durée de vie de la variable ou de la constante. Tenter d'affecter une valeur non-chaîne à la variable de nom déclarée ci-dessus entraînerait une erreur de compilation:

```
name = 3.10 // Error: "Cannot assign value of type 'Double' to 'String'"
```

Bien que Swift est un langage de type sécurisé, où les types de variables sont explicites et ne changent pas, il est possible de créer du code Swift qui se comporte comme un langage de type dynamique en utilisant le type de données Swift `Any`. Par exemple, le code suivant est légal dans Swift:

```
var anyType: Any  
anyType = "Ahmedi"  
anyType = 3.10
```

Mais ce n'est pas une bonne pratique de programmation Swift. Le type `Any` est principalement fourni pour permettre le passage entre le code Objective-C et Swift. vous devez utiliser des types explicites dans la mesure du possible, pour avoir un code sécurisé et sans erreur.

Comment nommer les variables en Swift

Les variables et constantes Swift ont les mêmes règles de dénomination que la plupart des langages inspirés du langage C :

- Ne doit pas commencer par un chiffre
- Après le premier caractère, les chiffres sont autorisés
- Peut commencer par un caractère de soulignement
- Les noms de symboles sont sensibles à la casse
- Les mots clés réservés peuvent être utilisés comme noms de variables s'ils sont inclus dans backticks (par exemple, `Int`: Int = 2021)

Contrairement à d'autres langages de programmation, Swift n'est pas limité à l'alphabet occidental pour ses caractères de nom de variable. Vous pouvez utiliser n'importe quel caractère Unicode dans le cadre de vos déclarations de variables. Les déclarations de variables suivantes sont acceptées dans Swift:

```
var 你好世界 = "Hello"  
var 😊 = "Smile!"
```

Tuples

L'une des fonctionnalités linguistiques uniques de Swift est son inclusion de tuples. Par défaut, les variables et les constantes stockent une valeur unique. Les tuples permettent à une variable ou à un nom de constante de faire référence à un ensemble de valeurs. Bien que les tuples n'existent pas dans de nombreux langages, vous pouvez les considérer comme des valeurs composées, et ils fonctionnent presque de la même manière qu'une structure, qui est un seul objet nommé qui peut stocker plus d'une variable.

Les tuples sont utiles pour créer une valeur composée - par exemple, pour retourner plusieurs valeurs d'une fonction. Les éléments d'un tuple se référencent soit par nombre ou par nom.

En utilisant un tuple, la déclaration de variable suivante:

```
var code = 404  
var message = "Not Found"
```

Pourrait être combinée à comme suit:

```
Var erreur = (404, "Not Found")
```

Nous accédons aux membres individuels du tuple comme suit:

```
print(erreur.0) // outputs 404  
print(erreur.1) // outputs Not Found
```

Les membres du tuple peuvent également avoir des noms individuels, comme suit:

```
Var erreur = (code:404, message:"Not Found")  
print(erreur.code) // outputs 404  
print(erreur.0) // also outputs 404  
print(erreur.message) // outputs Not Found
```

Les fonctions Swift peuvent accepter plusieurs paramètres d'entrée, mais ne renvoient qu'une seule valeur. Un cas d'utilisation courant pour un type de variable tuple est de retourner plusieurs par une fonction:

```
func getErreur() -> (code: Int, message: String) {  
    let erreur = (code: 404, message: "Not Found")  
    return erreur  
}  
Print(getErreur())
```

REMARQUE :

Les tuples sont utiles pour les groupes simples de valeurs liées. Ils ne sont pas adaptés à la création de structures de données complexes. Si votre structure de données est susceptible d'être plus complexe, modélisez-la comme une classe ou une structure plutôt qu'un tuple.

Valeur Optionnelle

Une valeur optionnelle contient soit une valeur, soit **nil** dans le cas où la valeur est manquante. Elle représente deux possibilités: soit il y a une valeur, et vous pouvez débiller l'optionnel pour accéder à cette valeur, soit il n'y a pas de valeur du tout. Ajoutez un point d'interrogation (?) après la déclaration du type d'une variable/constante pour la marquer comme optionnelle.

La variable **name** suivante n'est pas optionnelle:

```
var name: String = "Alami"
```

La variable **name** suivante est optionnelle et a une valeur initiale nil:

```
Var name: String?
```

L'exemple suivant montre la façon dont les options peuvent être utilisées pour faire face à l'absence de valeur. Le type **Int** de Swift a un initialiseur qui tente de convertir une valeur **String** en une valeur **Int**. Cependant, toutes les chaînes ne peuvent pas être converties en un entier. La chaîne "123" peut être convertie en valeur numérique 123, mais la chaîne "hello, world" n'est pas une valeur numérique à convertir.

```
let possibleNumber = "123"  
let convertedNumber = Int(possibleNumber)  
// convertedNumber est supposé être de type "Int?", or "optional Int"
```


La condition **if** et le déballage Forcé (**Forced Unwrapping**)

Vous pouvez utiliser une instruction **if** pour savoir si un optionnel contient une valeur en comparant le optionnel à **nil**. Vous effectuez cette comparaison avec l'opérateur «égal à» (**==**) ou l'opérateur «différent de» (**!=**).

Si un optionnel a une valeur, il est considéré comme "différent de" **nil**:

```
if convertedNumber != nil {  
    print("convertedNumber has an integer value of \(convertedNumber!).")  
}  
  
else {  
    print("The string \"\`(possibleNumber)`\" couldn't be converted to an integer")  
}  
  
// Prints "convertedNumber has an integer value of 123."
```

Vous pouvez combiner **if** et **let** pour traiter des valeurs qui peuvent être manquantes (optionnelles).

```
if let actualNumber = Int(possibleNumber) {  
    print("The string \"\`(possibleNumber)`\" has an integer value of  
    \(actualNumber) ")  
} else {  
    print("The string \"\`(possibleNumber)`\" couldn't be converted to an integer")  
}  
  
// Prints "The string "123" has an integer value of 123"
```

Si la valeur optionnelle vaut **nil**, alors la condition à vérifier est **false** et le code entre accolades **if** est ignoré. Dans le cas contraire, la valeur optionnelle est **déballée** (on parle de **:optional unwrapping**) et assignée à la constante après le mot-clé **let**, ce qui rend la valeur optionnelle disponible au sein du bloc de code.

Il est aussi possible de gérer les valeurs optionnelles en leur fournissant une valeur par défaut, grâce à l'opérateur `??`. Si la valeur optionnelle est manquante, alors la valeur par défaut sera utilisée à la place.

```
let nickname: String? = nil  
let fullName: String = "Ahmed Alami"  
let informalGreeting = "Salut \(nickname ?? fullName)"
```

III. Les fonctions

Les fonctions sont des blocs de code autonomes qui exécutent une tâche spécifique. On donne à une fonction un nom qui identifie ce qu'elle fait, et ce nom est utilisé pour «appeler» la fonction afin d'exécuter sa tâche en cas de besoin.

La syntaxe de fonction Swift est suffisamment flexible pour exprimer quoi que ce soit, d'une simple fonction de style C sans nom de paramètre à une méthode complexe de style Objective-C avec des noms et des étiquettes d'argument pour chaque paramètre. Les paramètres peuvent fournir des valeurs par défaut pour simplifier les appels de fonction et peuvent être passés en tant que paramètres d'entrée-sortie, qui modifient une variable transmise une fois que la fonction a terminé son exécution.

Chaque fonction Swift a un type, composé des types de paramètres de la fonction et du type de retour. Vous pouvez utiliser ce type comme tout autre type dans Swift, ce qui facilite la transmission de fonctions en tant que paramètres à d'autres fonctions et le retour de fonctions à partir de fonctions. Les fonctions peuvent également être écrites dans d'autres fonctions pour encapsuler des fonctionnalités utiles dans une portée de fonction imbriquée.

III.1 Définition et appel de fonctions

Lorsque vous définissez une fonction, vous pouvez éventuellement définir une ou plusieurs valeurs nommées et typées que la fonction prend en entrée, appelées **paramètres**. Vous pouvez également définir en option un type de valeur que la fonction transmettra en sortie une fois terminée, connu sous le nom de type de **retour**.

Pour utiliser une fonction, vous «appelez» cette fonction avec son nom et lui passez des valeurs d'entrée (appelées *arguments*) qui correspondent aux types de paramètres de la fonction. Les arguments d'une fonction doivent toujours être fournis dans le même ordre que la liste de paramètres de la fonction.

La fonction ci-dessous est appelée `greet(person:)`, elle prend le nom d'une personne comme entrée et renvoie un message d'accueil pour cette personne:

```
func greet(person: String) -> String {  
    let greeting = "Hello, " + person + "!"
```

```
    return greeting  
}
```

Le nom de la fonction est précédé du mot - clé **func**. Le type de retour de la fonction est précédé de l'opérateur **->**.

La définition permet d'appeler facilement la fonction sans ambiguïté dans votre code:

```
print(greet(person: "Ali"))  
// Prints "Bonjour, Ali!"  
print(greet(person: "Othmane"))  
// Prints " Bonjour, Othmane!"
```

La fonction `greet(person :)` est appelée en lui passant une valeur `String` après l'étiquette d'argument `person`, telle que `greet(person:"Ali")`. Étant donné que la fonction renvoie une valeur `String`, `greet(person:)` peut être encapsulé dans l'appel à la fonction `print(_:separator: terminator:)` pour afficher cette chaîne et voir sa valeur de retour.

Fonctions sans paramètres

N'est pas nécessaire de définir les paramètres d'entrée. Ci-dessous une fonction sans paramètres d'entrée, qui renvoie toujours le même message `String` à chaque fois qu'elle est appelée:

```
func sayHelloWorld() -> String {  
    return "hello, world"  
}  
  
print(sayHelloWorld())  
// Prints "hello, world"
```

La définition de la fonction a toujours besoin de parenthèses après le nom de la fonction, même si elle ne prend aucun paramètre. Le nom de la fonction est également suivi d'une paire de parenthèses vides lorsque la fonction est appelée.

Fonctions avec plusieurs paramètres

Les fonctions peuvent avoir plusieurs paramètres d'entrée, qui sont écrits entre parenthèses de la fonction, séparés par des virgules.

Cette fonction prend le nom d'une personne et si elle a déjà été accueillie comme entrée, et renvoie un message d'accueil approprié pour cette personne:

```
func greet(person: String, alreadyGreeted: Bool) -> String {  
  if alreadyGreeted {  
    return greetAgain(person: person)  
  } else {  
    return greet(person: person)  
  }  
}  
print(greet(person: "Ali", alreadyGreeted: true))  
// Prints "Hello again, Ali!"
```

Fonctions sans valeurs de retour

Les fonctions peut ne pas avoir un type de retour. Ci-dessous une version de la fonction `greet(person:)`, qui affiche sa propre valeur `String` plutôt que de la renvoyer:

```
func greet(person: String) {  
  print("Hello, \ (person)!")  
}  
greet(person: "ALi")  
// Prints "Hello, Ali!"
```

Comme il n'est pas nécessaire de renvoyer une valeur, la définition de la fonction n'inclut pas la flèche de retour (`->`) ou un type de retour.

Fonctions avec plusieurs valeurs de retour

pour une fonction pour renvoyer plusieurs valeurs dans le cadre d'une valeur de retour composée ,on utilise un type tuple comme type de retour.

L'exemple ci-dessous définit une fonction appelée `minMax(array:)`, qui recherche les nombres les plus petits et les plus grands dans un tableau de valeurs `Int`:

```
func minMax(array: [Int]) -> (min: Int, max: Int) {  
  var currentMin = array[0]  
  var currentMax = array[0]  
  for value in array[1..  
array.count] {  
    if value < currentMin {  
      currentMin = value  
    } else if value > currentMax {  
      currentMax = value  
    }  
  }  
  return (currentMin, currentMax)  
}
```

La fonction `minMax(array:)` renvoie un tuple contenant deux valeurs `Int`. Ces valeurs sont étiquetées `min` et `max` afin d'être accessibles par leurs noms .

```
let chiffres = minMax(array: [8, 2, -1, 2, 19, 3, 97])
print("min is \(chiffres.min) and max is \(chiffres.max)")
// Prints "min is -6 and max is 97"
```

Types de retour : tuple optionnel

Si le type de tuple à renvoyer à partir d'une fonction a le potentiel de n'avoir «aucune valeur» pour le tuple entier, vous pouvez utiliser un type de retour de tuple optionnel pour refléter le fait que le tuple entier peut l'être nil. Vous écrivez un type de retour de tuple optionnel en plaçant un point d'interrogation après la parenthèse du tuple, tel que (Int, Int)? ou (String, Int, Bool)?

La fonction minMax (array :) ci-dessus renvoie un tuple contenant deux valeurs Int. Cependant, la fonction n'effectue aucun contrôle de sécurité sur la baie à laquelle elle a réussi. Si l'argument tableau contient un tableau vide, la fonction minMax (array :), telle que définie ci-dessus, déclenchera une erreur d'exécution lors de la tentative d'accès au tableau [0].

La fonction minMax(array:) ci-dessus renvoie un tuple contenant deux valeurs Int. Cependant, la fonction n'effectue aucun contrôle de sécurité sur le tableau reçu. Si l'argument array contient un tableau vide, la minMax(array:), telle que définie ci-dessus, déclenchera une erreur d'exécution lors de la tentative d'accès array[0].

Pour gérer un tableau vide en toute sécurité, écrivez la minMax(array:) fonction avec un type de retour de tuple facultatif et retournez une valeur de nil lorsque le tableau est vide:

```
func minMax(array: [Int]) -> (min: Int, max: Int)? {
    if array.isEmpty { return nil }
    var currentMin = array[0]
    var currentMax = array[0]
    for value in array[1..
```

Vous pouvez vérifier si cette version de la fonction minMax(array:) renvoie une valeur de tuple réelle ou nil:

```
if let bounds = minMax(array: [8, -6, 2, 109, 3, 71]) {
    print("min is \(bounds.min) and max is \(bounds.max)")
}
```

```
}  
// Prints "min is -6 and max is 109"
```

Fonctions avec un retour implicite

Si le corps entier de la fonction est une seule expression, la fonction renvoie implicitement cette expression. Par exemple, les deux fonctions ci-dessous ont le même comportement:

```
func greeting(for person: String) -> String {  
    "Hello, " + person + "!"  
}  
print(greeting(for: "Karim"))  
// Prints "Hello, Karim!"  
  
func anotherGreeting(for person: String) -> String {  
    return "Hello, " + person + "!"  
}  
print(anotherGreeting(for: " Karim "))  
// Prints "Hello, Karim!"
```

La définition complète de la `greeting(for:)` fonction est le message d'accueil qu'elle renvoie, ce qui signifie qu'elle peut utiliser cette forme plus courte.

La `anotherGreeting(for:)` fonction renvoie le même message d'accueil, en utilisant le mot clé `return` comme une fonction plus longue. Toute fonction que vous écrivez sur une seule ligne peut omettre le `return`.

Étiquettes d'argument et noms de paramètre

Chaque paramètre de fonction a à la fois une *étiquette d'argument* et un *nom de paramètre*. L'étiquette d'argument est utilisée lors de l'appel de la fonction; chaque argument est écrit dans l'appel de fonction avec son étiquette d'argument devant lui. Le nom du paramètre est utilisé dans l'implémentation de la fonction. Par défaut, les paramètres utilisent leur nom de paramètre comme étiquette d'argument.

```
func someFunction(firstParameterName: Int, secondParameterName: Int) {  
    // Dans le corps de la fonction, firstParameterName et secondParameterName
```

```
// font référence aux valeurs d'argument pour les premier et deuxième
paramètres.
}
someFunction(firstParameterName: 1, secondParameterName: 2)
```

Spécification des étiquettes d'argument

Vous écrivez une étiquette d'argument avant le nom du paramètre, séparé par un espace:

```
func someFunction(argumentLabel parameterName: Int) {
    // Dans le corps de la fonction, parameterName fait référence à la valeur
    //d'argument.
}
```

La fonction `greet(person:)` prend le nom et la ville d'une personne et renvoie un message d'accueil:

```
func greet(person: String, from hometown: String) -> String {
    return "Hello \(person)! Glad you could visit from \(hometown)."
}
print(greet(person: "Bill", from: "Cupertino"))
// Prints "Hello Bill! Glad you could visit from Cupertino."
```

L'utilisation d'étiquettes d'argument peut permettre d'appeler une fonction d'une manière expressive, semblable à une phrase, tout en fournissant un corps de fonction lisible et clair dans l'intention.

Omettre les étiquettes d'argument

Si vous ne voulez pas d'étiquette d'argument pour un paramètre, écrivez un trait de soulignement (`_`) au lieu d'une étiquette d'argument explicite pour ce paramètre.

```
func someFunction(_ firstParameterName: Int, secondParameterName: Int) {
    // In the function body, firstParameterName and secondParameterName
    // refer to the argument values for the first and second parameters.
}
```

```
someFunction(1, secondParameterName: 2)
```

Si un paramètre a une étiquette d'argument, l'argument doit être étiqueté lorsque vous appelez la fonction.

Valeurs des paramètres par défaut

Vous pouvez définir une valeur par défaut pour n'importe quel paramètre d'une fonction en attribuant une valeur au paramètre après le type de ce paramètre. Si une valeur par défaut est définie, vous pouvez omettre ce paramètre lors de l'appel de la fonction.

```
func someFunction(parameterWithoutDefault: Int, parameterWithDefault: Int =  
12) {  
    // Si vous omettez le deuxième argument lors de l'appel de cette fonction,  
    //alors la valeur de parameterWithDefault sera 12.  
}  
  
someFunction(parameterWithoutDefault: 3, parameterWithDefault: 6) //  
parameterWithDefault is 6  
  
someFunction(parameterWithoutDefault: 4) // parameterWithDefault is 12
```

Paramètres variadiques

Un paramètre variadique accepte zéro ou plusieurs valeurs d'un type spécifié. Un paramètre variadique est utilisé pour spécifier que le paramètre peut recevoir un nombre variable de valeurs d'entrée lorsque la fonction est appelée. Ecrivez des paramètres variadiques en insérant trois points (...) après le type du paramètre.

Les valeurs passées à un paramètre variadique sont rendues disponibles dans le corps de la fonction sous la forme d'un tableau du type approprié. Par exemple, un paramètre variadique avec un nom `numbers` et un type de `Double`...est rendu disponible dans le corps de la fonction sous la forme d'un tableau constant appelé `numbers` de type `[Double]`.

L'exemple ci-dessous calcule la moyenne pour une liste de nombres de n'importe quelle longueur:

```
func arithmeticMean(_ numbers: Double...) -> Double {  
    var total: Double = 0  
    for number in numbers {  
        total += number  
    }  
}
```



```
    return total / Double(numbers.count)
}
arithmeticMean(1, 2, 3, 4, 5)
// retourne 3.0, t la moyenne arithmétique des cinq nombres
arithmeticMean(3, 8.25, 18.75)
// retourne 10.0, t la moyenne arithmétique de dix nombres
```

Paramètres d'entrée-sortie

Les paramètres de fonction sont des constantes par défaut. La modification de la valeur d'un paramètre à partir du corps de cette fonction entraîne une erreur de compilation. Cela signifie qu'on peut pas modifier la valeur d'un paramètre par erreur. Si on souhaite qu'une fonction modifie la valeur d'un paramètre et qu'on souhaite que ces modifications persistent après l'appel de la fonction, il faut définir ce paramètre en tant que paramètre d'entrée-sortie (*in-out*).

Un paramètre d'entrée-sortie est défini en plaçant le mot clé `inout` juste avant le type du paramètre. Un paramètre d'entrée-sortie a une valeur qui est transmise à la fonction, est modifiée par la fonction et est renvoyée hors de la fonction pour remplacer la valeur d'origine.

Vous pouvez seulement passer une variable comme argument d'un paramètre d'entrée-sortie. Vous ne pouvez pas passer une constante ou une valeur littérale comme argument, car les constantes et les littéraux ne peuvent pas être modifiés. Vous placez (`&`) directement avant le nom d'une variable lorsque vous la transmettez en tant qu'argument à un paramètre d'entrée-sortie, pour indiquer qu'elle peut être modifiée par la fonction.

***** Les paramètres d'entrée-sortie ne peuvent pas avoir de valeurs par défaut et les paramètres variadiques ne peuvent pas être marqués comme `inout`.**

Exemple de fonction appelée `swapTwoInts(_:_:)`, qui a deux paramètres entiers d'entrée-sortie appelés `a` et `b`:

```
func swapTwoInts(_ a: inout Int, _ b: inout Int) {
    let temporaryA = a
    a = b
    b = temporaryA
}
```

La fonction `swapTwoInts(_:_:)` permute les deux valeurs de `a` et `b`. La fonction effectue cette permutation en stockant la valeur de `a` dans une constante temporaire appelée `temporaryA`, en affectant la valeur de `b` à `a`, puis en affectant `temporaryA` à `b`.

```
var someInt = 3
var anotherInt = 107
swapTwoInts(&someInt, &anotherInt)
print("someInt is now \(someInt), and anotherInt is now \(anotherInt)")
// Prints "someInt is now 107, and anotherInt is now 3"
```

L'exemple ci-dessus montre que les valeurs d'origine de `someInt` et `anotherInt` sont modifiées par la `swapTwoInts(_:_:)` fonction, même si elles ont été définies à l'origine en dehors de la fonction.

Types de fonction

Chaque fonction a un type spécifique, composé des types de paramètres et du type de retour de la fonction.

Par exemple:

```
func addTwoInts(_ a: Int, _ b: Int) -> Int {
    return a + b
}

func multiplyTwoInts(_ a: Int, _ b: Int) -> Int {
    return a * b
}
```

Cet exemple définit deux fonctions mathématiques simples `addTwoInts` et `multiplyTwoInts`. Ces fonctions prennent chacune deux valeurs `Int` et renvoient une valeur `Int`, qui est le résultat de l'exécution d'une opération mathématique appropriée.

Le type de ces deux fonctions est `(Int, Int) -> Int`. Cela peut être lu comme

"Une fonction qui a deux paramètres, les deux de type `Int`, et qui renvoie une valeur de type `Int`."

Utilisation des types de fonction

Vous utilisez des types de fonction comme tous les autres types dans Swift. Par exemple, vous pouvez définir une constante ou une variable comme étant de type fonction et affecter une fonction appropriée à cette variable:

```
var mathFunction: (Int, Int) -> Int = addTwoInts
```

Cela peut être lu comme:

«Définissez une variable appelée `mathFunction`, qui est un type de 'une fonction qui prend deux valeurs `Int` et renvoie une valeur `Int`.' Définissez cette nouvelle variable pour faire référence la fonction `addTwoInts`. »

La fonction `addTwoInts(_:_:)` a le même type que la variable `mathFunction`, et donc cette affectation est autorisée par le vérificateur de type de Swift.

Vous pouvez maintenant appeler la fonction attribuée au nom `mathFunction`:

```
print("Result: \(mathFunction(2, 3))")  
// Prints "Result: 5"
```

Une autre fonction avec même type peut être affectée à la même variable:

```
mathFunction = multiplyTwoInts  
print("Result: \(mathFunction(2, 3))")  
// Prints "Result: 6"
```

Comme pour tout autre type, le Swift déduit le type de fonction lorsqu'elle est affectée à une constante ou une variable:

```
let anotherMathFunction = addTwoInts  
// anotherMathFunction est supposé être de type (Int, Int) -> Int
```

Types de fonction en tant que types de paramètres

Vous pouvez utiliser un type de fonction tel que `(Int, Int) -> Int` comme type de paramètre pour une autre fonction. Cela il permet de laisser certains aspects de l'implémentation d'une fonction à l'appelant de la fonction.

Un exemple pour imprimer les résultats des fonctions mathématiques ci-dessus:

```
func printMathResult(_ mathFunction: (Int, Int) -> Int, _ a: Int, _ b: Int) {  
    print("Result: \(mathFunction(a, b))")  
}
```

```
}  
  
printMathResult(addTwoInts, 3, 5)  
  
// Prints "Result: 8"
```

Cet exemple définit une fonction appelée `printMathResult(_:_:_:)`, qui a trois paramètres. Le premier paramètre est appelé `mathFunction` et est de type `(Int, Int) -> Int`. Vous pouvez passer n'importe quelle fonction de ce type comme argument pour ce premier paramètre. Les deuxième et troisième paramètres sont `a` et `b` les deux de type `Int`. Celles-ci sont utilisées comme deux valeurs d'entrée pour la fonction mathématique fournie.

Cet exemple définit la fonction `printMathResult(_:_:_:)`, qui comporte trois paramètres. Le premier paramètre est appelé `mathFunction` et est de type `(Int, Int) -> Int`. Vous pouvez passer toute fonction ayant ce type comme argument pour ce premier paramètre. Les deuxième et troisième paramètres sont appelés `a` et `b` et sont de type `Int`, qui sont utilisés comme valeurs d'entrée pour la fonction mathématique fournie.

Lorsqu'il `printMathResult(_:_:_:)` est appelé, il est passé la `addTwoInts(_:_:_:)` fonction, et les valeurs entières 3 et 5. Il appelle la fonction fournie avec les valeurs 3 et 5, et imprime le résultat de 8.

Types de fonction comme types de retour

Vous pouvez utiliser un type de fonction comme type de retour d'une autre fonction. Pour ce faire, on écrit un type de fonction complet immédiatement après la flèche de retour (`->`) du retour de la fonction.

L'exemple suivant définit deux fonctions simples appelées `stepForward(_:_:)` et `stepBackward(_:_:)`. La fonction `stepForward(_:_:)` renvoie une valeur un de plus que sa valeur d'entrée, et la fonction `stepBackward(_:_:)` renvoie une valeur un de moins que sa valeur d'entrée. Les deux fonctions ont un type de `(Int) -> Int`:

```
func stepForward(_ input: Int) -> Int {  
    return input + 1  
}  
  
func stepBackward(_ input: Int) -> Int {  
    return input - 1  
}
```

La fonction `chooseStepFunction (backward :)`, dont le type de retour est `(Int) -> Int`.

La fonction `chooseStepFunction (backward :)` renvoie la fonction `stepForward (_:_:)` ou la fonction `stepBackward (_:_:)` basée sur un paramètre booléen appelé `backward`:

```
func chooseStepFunction(backward: Bool) -> (Int) -> Int {  
    return backward ? stepBackward : stepForward  
}
```

On peut utiliser `chooseStepFunction(backward:)` pour obtenir une fonction qui avancera dans un sens ou dans l'autre:

```
var currentValue = 3  
let moveNearerToZero = chooseStepFunction(backward: currentValue > 0)  
// moveNearerToZero reference la fonction stepBackward()
```

L'exemple ci-dessus détermine si un pas positif ou négatif est nécessaire pour rapprocher progressivement une variable appelée `currentValue` de zéro. `currentValue` a une valeur initiale de 3, ce qui signifie que `currentValue > 0` renvoie `true`, ce qui oblige `chooseStepFunction(backward:)` à renvoyer la fonction `stepBackward` (`_:`). Une référence à la fonction retournée est stockée dans une constante appelée `moveNearerToZero`. Maintenant que `moveNearerToZero` fait référence la fonction correcte, il peut être utilisé pour compter jusqu'à zéro:

```
print("Counting to zero:")  
// Counting to zero:  
while currentValue != 0 {  
    print("\(currentValue)... ")  
    currentValue = moveNearerToZero(currentValue)  
}  
print("zero!")  
// 3...  
// 2...  
// 1...  
// zero!
```

Fonctions imbriquées

Toutes les fonctions définies jusqu'à présent sont des *fonctions globales*, qui sont définies à une portée globale. Vous pouvez également définir des fonctions à l'intérieur d'autres fonctions, appelées *fonctions imbriquées*.

Les fonctions imbriquées sont cachées du monde extérieur par défaut, mais peuvent toujours être appelées et utilisées par leur fonction englobante. Une fonction englobante peut également renvoyer l'une de ses fonctions imbriquées pour permettre à la fonction imbriquée d'être utilisée dans une autre portée.

La fonction `chooseStepFunction(backward:)` utilise et renvoie des fonctions imbriquées:

```
func chooseStepFunction(backward: Bool) -> (Int) -> Int {  
    func stepForward(input: Int) -> Int { return input + 1 }  
    func stepBackward(input: Int) -> Int { return input - 1 }  
    return backward ? stepBackward : stepForward  
}  
  
var currentValue = -3  
  
let moveNearerToZero = chooseStepFunction(backward: currentValue > 0)  
  
// moveNearerToZero reference la fonction imbriquée stepForward()  
  
while currentValue != 0 {  
    print("\(currentValue) ... ")  
    currentValue = moveNearerToZero(currentValue)  
}  
  
print("zero!")  
  
    // -3...  
    // -2...  
    // -1...  
    // zero!
```

IV. Fermetures

Une fermeture (en anglais closures) en Swift permet d'utiliser une fonction sans l'avoir déclaré avant. Une fermeture, c'est une fonction qui n'a pas de nom. Elle représente un bloc de fonctionnalités autonome qui peut être transmis et utilisé dans votre code. Les fermetures dans Swift sont similaires aux blocs en C et Objective-C et aux lambdas dans d'autres langages de programmation.

Les fermetures peuvent capturer et stocker des références à toutes les constantes et variables du contexte dans lequel elles sont définies. C'est ce qu'on appelle la *fermeture* de ces constantes et variables.

Les fonctions globales et imbriquées sont en fait des cas particuliers de fermetures. Les fermetures prennent l'une des trois formes suivantes:

- Les fonctions globales sont des fermetures qui ont un nom et ne capturent aucune valeur.
- Les fonctions imbriquées sont des fermetures qui ont un nom et peuvent capturer des valeurs à partir de leur fonction englobante.
- Les expressions de fermeture sont des fermetures sans nom écrites dans une syntaxe légère qui peut capturer des valeurs à partir de leur contexte environnant.

Dans la section précédente, vous avez appris à passer une fonction nommée à une autre fonction, permettant à cette dernière d'appeler la première au moment opportun. Les fermetures sont un autre moyen de passer du code à une fonction.

Syntaxe d'expression de fermeture

La syntaxe de l'expression de fermeture a la forme générale suivante:

```
{ (parameters) -> return type in  
  statements  
}
```

Une fermeture commence par une accolade. Ensuite on lui passe les paramètres et les valeurs de retour comme une fonction normale. Ensuite on ajoute le mot clé **in**. Et ensuite vient le corps de la fermeture terminé par une accolade.

Les *paramètres* de la syntaxe de l'expression de fermeture peuvent être des paramètres d'entrée-sortie, mais ils ne peuvent pas avoir de valeur par défaut. Les paramètres variadiques peuvent être utilisés si vous nommez le paramètre variadique. Les tuples peuvent également être utilisés comme types de paramètres et types de retour.

Exemple 1 :

```
let maClosure = { print( "Bonjour, Je suis une fermeture") }  
maClosure() // le code est exécuté, il affiche le message :  
            //    Bonjour, Je suis une fermeture
```

Exemple 2 :

```
let complexClosure = {(name:String, age:Float) -> Bool in  
  if age > 18 {return true}}
```

```
    else {return false }  
  }  
  let success = complexClosure("Louis",32)  
  print("Louis has \"(success)\"")
```

L'exemple ci-dessous montre une expression de fermeture de tri décroissant du tableau `names`:

```
let names = ["Hamin", "Alami", "Karam", "Badri", "Dami"]  
décroissantNames = names.sorted(by: { (s1: String, s2: String) -> Bool in  
  return s1 > s2  
})
```